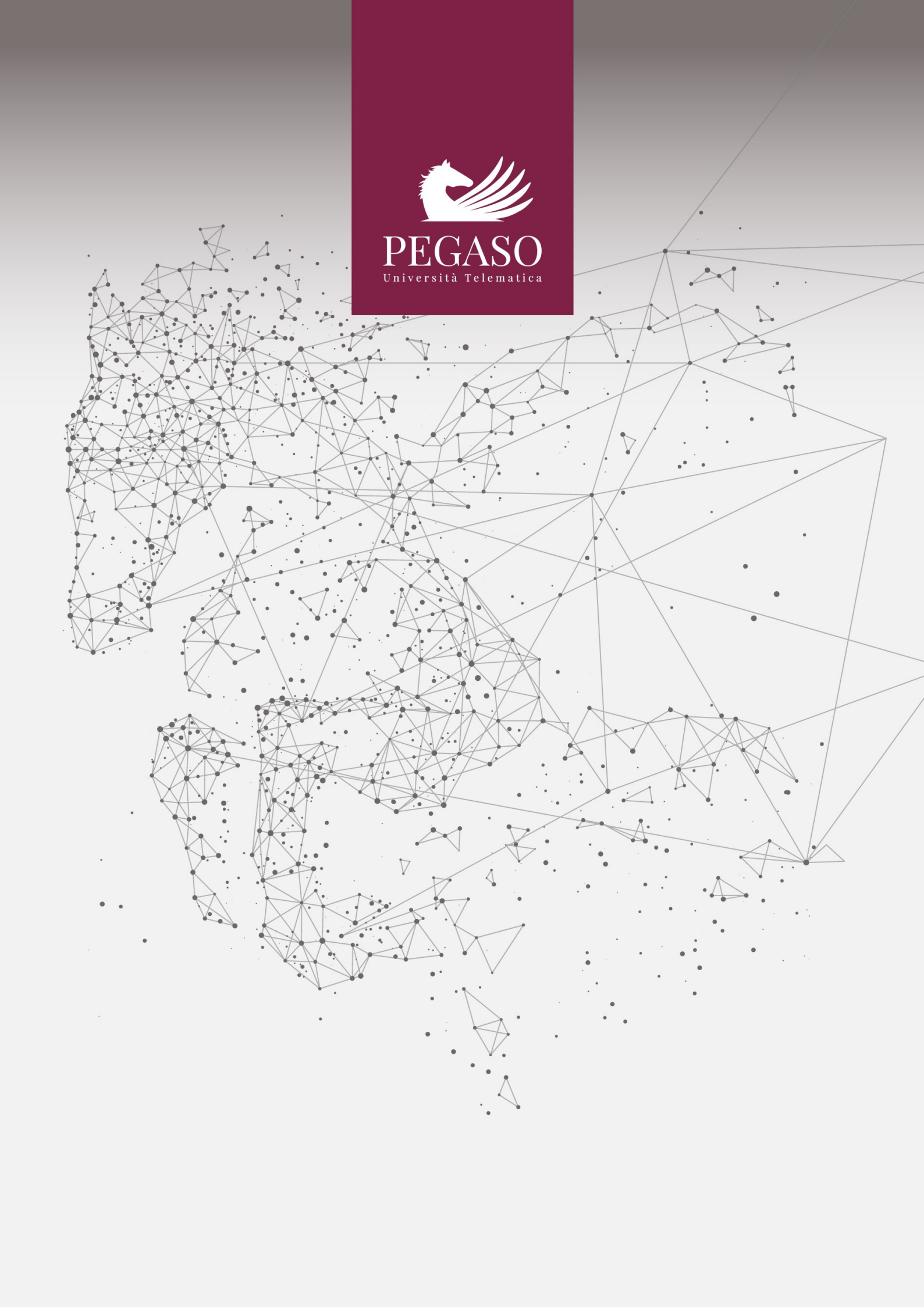

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. RICHIAMO DELLE NOZIONI RELATIVE ALLE OPERAZIONI DI ACCESSO IN MEMORIA 3

2. CALCOLO DELL'INDIRIZZO DELL'ELEMENTO A[K] DI UN ARRAY 6

3. STRINGHE COME ARRAY DI CARATTERI.....9

BIBLIOGRAFIA 12

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Richiamo delle nozioni relative alle operazioni di accesso in Memoria

La struttura dati Array è una lista di elementi in cui ogni elemento è direttamente accessibile tramite un *indice* ad esso associato. Gli elementi di un Array sono tutti dello stesso Tipo e la *dimensione* dell'Array stabilisce il numero degli elementi che lo costituiscono e lo spazio di Memoria assegnato all'Array in fase di compilazione. L'indice varia da 0 al valore della *dimensione* meno 1.

I Linguaggi ad Alto Livello che supportano il Tipo di dati Array mettono a disposizione gli operatori necessari per la gestione degli elementi, in particolare, per la lettura e la scrittura in Memoria di un elemento dell'Array specificato dal relativo indice.

Nei **Linguaggi C, C++ e Java** l'operatore che consente l'accesso allo specifico elemento di un Array è denotato dalle parentesi quadre, quindi le semplici istruzioni di assegnamento:

val = A[k] *lettura* dell'elemento A[k] in Memoria

A[k] = val *scrittura* dell'elemento A[k] in Memoria

consento, rispettivamente, l'accesso in Memoria in lettura ed in scrittura per la gestione diretta dell'elemento dell'Array il cui indice è dato dal valore della variabile k.

Il **Linguaggio Assembly** NON DISPONE di operatori come le parentesi quadre per gestire l'accesso in Memoria per riferimento mediante l'indice. Occorre fornire l'indirizzo di Memoria relativo all'elemento, sia nel caso che il valore dell'indice sia dato esplicitamente mediante una costante numerica, sia nel caso che sia fornito dal valore di una variabile. Gli accessi in Memoria sono effettuati mediante le istruzioni di trasferimento dati **LOAD** e **STORE**, riferite a dati rappresentati da sequenze di 8 bit (*Byte*), 16 bit (*mezza parola*), 32 bit (*parola*).

Le regole sintattiche e la Modalità di Indirizzamento sono del tutto analoghe, cambia il Tipo di dato e l'accesso in Memoria che è in LETTURA per *Load* ed in SCRITTURA *per Store*.

Ricordiamo brevemente le regole sintattiche e la Modalità di Indirizzamento considerando come esempio l'istruzione *load word*.

Istruzione Assembly MIPS **load word**:

lw $t1, 3 ($s0)

LETTURA in Memoria ⇒ scrittura in un registro.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Vincoli sintattici:

- Codice Operativo **lw** (**l**oad **w**ord) che specifica l'operazione,
- dopo il Codice Operativo viene indicato l'**indirizzo del registro,**

in cui viene scritta la copia dalla *parola* letta dalla Memoria,

- segue un numero intero detto **OFFSET** (spiazzamento),
- in parentesi è indicato l'**indirizzo del registro** detto **REGISTRO BASE** che contiene un **indirizzo di Memoria.**

Per l'istruzione *Store* cambia il significato dell'operazione di trasferimento effettuata:

Istruzione Assembly MIPS **store word**

sw $t1, 3 ($s0)

lettura di un registro ⟹ **SCRITTURA in Memoria**.

Le istruzioni relative alle varie lunghezze delle sequenze sono:

sequenza binaria trasferita	**LOAD**	**STORE**
una parola (4 Byte) (4 locazioni)	**lw**	**sw**
mezza parola (2 Byte) (2 locazioni)	**lh**	**sh**
mezza parola (2 Byte) (2 locazioni)	*unsigned*	**lhu**
8 bit (un Byte) (una locazioni)	**lb**	**sb**
8 bit (un Byte) (una locazioni)	*unsigned*	**lbu**

Nel caso di *lw* il dato letto è una *parola* quindi vengono lette 4 locazioni di Memoria consecutive (4 Byte), che costituiscono la sequenza di 32 bit da copiare nel registro occupandone tutti i bit.

Nel caso delle istruzioni *lh, lhu, lb, lbu* le locazioni consecutive lette sono 2 (2 Byte) oppure una (un Byte).

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quindi la scrittura della sequenza nel registro lascia libere delle posizioni, ed è richiesto un COMPLETAMENTO per ottenere una rappresentazione equivalente del dato con 32 bit, che si effettua con due diverse modalità:

- *lh* e *lb* per numeri interi effettuano l'***estensione del segno*** (copiando il valore del bit di segno),
- *lhu* e *lbu* per numeri *unsigned* aggiungono **bit uguali a 0**.

Per le istruzioni STORE non ci sono istruzioni per dati *unsigned* in quanto *sh* e *sb* effettuano la scrittura della sequenza binaria in Memoria nelle cui locazioni non avvengono ulteriori elaborazioni che possono dipendere dalla interpretazione del valore come numero con segno oppure *unsigned*, come invece avviene quando il dato viene scritto nei registri del processore.

Inoltre, non è necessario completare la sequenza scritta in quanto il dato scritto in Memoria nel caso di *sh* e *sb* occupa una o due locazioni senza lasciare bit vuoti da riempire.

L'Offset ed il Registro Base vengono utilizzati per il calcolo dell'INDIRIZZO DI MEMORIA da cui inizia il PRIMO BYTE della rappresentazione del dato da trasferire (approccio ***Big endian***).

La Modalità di Indirizzamento utilizzata per il calcolo di tale indirizzo è detta ***Indirizzamento tramite Base e Offset***.

INDIRIZZAMENTO TRAMITE BASE E OFFSET

Indirizzo di Memoria =

contenuto del *Registro Base* + *Offset* esteso di segno a 32 bit

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Calcolo dell'indirizzo dell'elemento A[k] di un Array

Poiché la GESTIONE DI UN ARRAY prevede accessi in Memoria in lettura e scrittura, nella **traduzione in Assembly** sono coinvolte le istruzioni di trasferimento dati *Load* e *Store*.

Questo porta ad assumere l'indirizzo del PRIMO BYTE del PRIMO ELEMENTO di un Array come INDIRIZZO BASE per gli accessi in Memoria utilizzando l'OFFSET per la gestione degli indici dell'Array.

Quando un elemento dell'Array è rappresentato mediante un **INDICE COSTANTE** la Modalità di Indirizzamento di *Load* e *Store* consente di ottenere l'indirizzo di accesso in Memoria semplicemente indicando un OPPORTUNO VALORE dell'*Offset*.

Poiché tutti gli elementi dell'Array sono dello stesso Tipo di dato, il valore dell'*Offset* è ottenuto moltiplicando il numero di Byte necessari a memorizzare un elemento per il valore dell'indice:

OFFSET = dimensione dell'elemento dell'Array x indice.

L'indirizzo completo viene calcolato dall'Hardware del computer automaticamente durante l'esecuzione dell'istruzione di trasferimento dati.

ESEMPIO Array A[ ] di numeri interi, INDICE COSTANTE

```
val = A[5] ;
A[0] = val ;
```

scegliendo i registri:

Base A [ ]	val
$s0	$t1

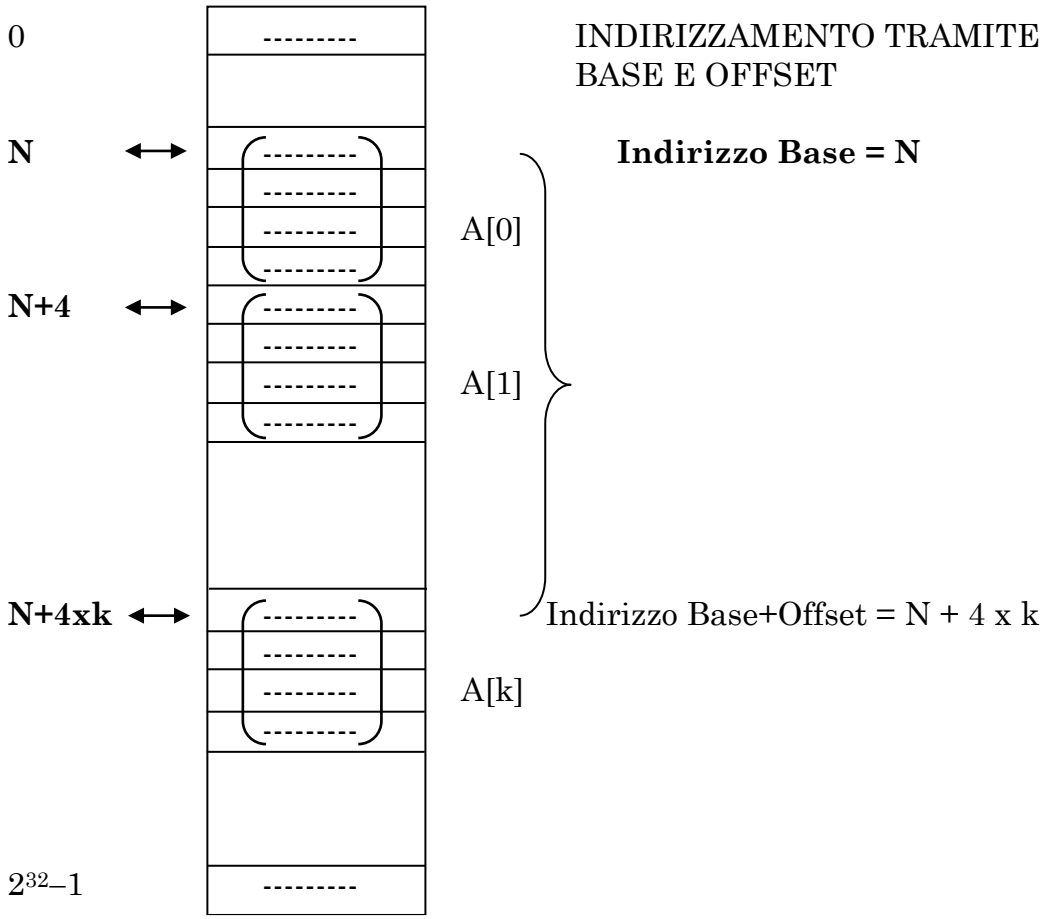
traduzione in Assembly MIPS:

```
lw  $t1, 20 ($s0)     # val = A[5]   Offset = 4 x 5 = 20
sw  $t1,  0 ($s0)     # A[0] = val   Offset = 4 x 0 = 0.
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il seguente schema fornisce una rappresentazione grafica della regola per la gestione degli elementi di un Array riferita al caso di un Array di interi, in cui ogni elemento occupa 4 locazioni di Memoria:



Quando, invece, l'indice dell'elemento dell'Array è dato dal valore di una **VARIABILE**, bisogna effettuare il CALCOLO DELL'INDIRIZZO COMPLETO mediante istruzioni Assembly applicando la regola della Modalità di Indirizzamento tramite Base e Offset. Le operazioni che è necessario realizzare sono le seguenti:

CALCOLO DELL'INDIRIZZO COMPLETO

- **Calcolo del valore dell'Offset** moltiplicando per 4, per 2, per 1 il valore della variabile che rappresenta l'indice a seconda che il dato sia rappresentato da una *parola*, da *mezza parola*, da un *Byte*.

  La moltiplicazione per 4 o per 2 può essere eseguita utilizzando, rispettivamente, l'istruzione *sll* di *shift logico a sinistra* di 2 o 1 posizione.
- **Calcolo dell'indirizzo completo** addizionando il valore ottenuto dell'*Offset* con l'*indirizzo Base dell'Array*, e memorizzando il risultato in un registro di appoggio;

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

l'**indirizzo di accesso in Memoria** così calcolato viene utilizzato in istruzioni *Load* o *Store* ponendo il registro di appoggio che lo contiene come registro Base e indicando l'Offset uguale a 0.

ESEMPIO Array A[ ] di numeri interi, INDICE VARIABILE

val = A[k] ;

A[k] = val ;

scegliendo i registri:

Base A [ ]	val	k
$s0	$t1	$t3

la traduzione in Assembly MIPS è:

```
sll  $t5,  $t3,  2        #  $t5  contiene l'Offset = 4 x k
add  $t6,  $t5, $s0       #  $t6 contiene l'indirizzo completo di A[k]
lw   $t1,  0 ($t6)        #  val = A[k]
sw   $t1,  0 ($t6)        #  A[k] = val
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Stringhe come Array di caratteri

Il Tipo di dato **stringa** è una struttura dati costituita da una sequenza di caratteri e da operatori che effettuano elaborazioni come la concatenazione di stringhe ed il confronto.

Anche se non è un Tipo di dato primitivo, i Linguaggi ad Alto Livello ne consentono la gestione con modalità differenti.

Il Linguaggio C implementa il Tipo di dato *stringa* mediante un ARRAY di caratteri ASCII, che sono rappresentati mediante 8 bit. L'ultimo elemento della *stringa* è il carattere NULL che ha codice 0.

Poiché lo standard ASCII assegna codice non nullo 48 al carattere 0, è possibile avere all'interno della *stringa* anche caratteri 0 senza possibilità di confusione con il carattere di fine *stringa* NULL.

Le istruzioni Assembly MIPS di trasferimento dati usate per la gestione delle *stringhe* di caratteri con codifica ASCII sono *Load Byte unsigned* (lbu) e *Store Byte* (sb).

Per il calcolo dell'Offset bisogna tenere conto del fatto che un elemento di un Array che implementa una *stringa* di caratteri **ASCII** è memorizzato in **UNA SOLA locazione di Memoria** (un Byte). Quindi, in questo caso, l'Offset è uguale al valore dell'indice,

ESEMPIO Array testo[ ] di caratteri ASCII, INDICE COSTANTE:

val = testo[5] ;

scegliendo i registri:

Base testo[ ]	val
$s0	$t1

la traduzione in Assembly MIPS è:

```
lbu  $t1,  5 ($s0)          #   val = testo[5]     Offset = 1 x 5 = 5
```

Quando, invece, un elemento di una *stringa* è indicato mediante un indice VARIABILE, bisogna calcolare l'indirizzo completo di accesso in Memoria applicando la regola della Modalità di Indirizzamento tramite Base e Offset:

CALCOLO DELL'INDIRIZZO COMPLETO

- **Calcolo del valore dell'Offset:**

  Offset = dimensione dell'elemento x indice = 1 x indice = indice

  poiché il codice di un carattere ASCII occupa una locazione.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Calcolo dell'indirizzo completo** addizionando il valore ottenuto dell'*Offset* con l'*indirizzo Base dell'Array*.

L'**indirizzo completo** così calcolato viene utilizzato in istruzioni *Load* o *Store* ponendo il registro che lo contiene come registro Base e indicando l'Offset uguale a 0.

ESEMPIO Array testo[ ] di caratteri ASCII, INDICE VARIABILE:

val = testo[k] ;

testo[k] = val ;

scegliendo i registri:

Base testo[ ]	val	k
$s0	$t1	**$t3**

la traduzione in Assembly MIPS è:

```
add  $t6,  $t3, $s0     # $t6 contiene l'indirizzo completo di testo[k]
                        #  in questo caso   Offset = 1 x k = k
lbu  $t1,  0 ($t6)      #   val = testo[k]
sb  $t1,  0 ($t6)       #   testo[k] = val
```

Nel caso di un Array di caratteri con codifica **Unicode UTF-16** ogni elemento dell'Array occupa **DUE LOCAZIONI di Memoria**. Quindi, le istruzioni Assembly MIPS di trasferimento dati usate per la gestione delle *stringhe* di caratteri con codifica Unicode a 16 bit sono *Load half word unsigned* (lhu) e *Store half word* (sh), rispettivamente per l'accesso alla Memoria in lettura e in scrittura.

In questo caso per il calcolo dell'indirizzo dell'elemento testo[k] occorre moltiplicare per 2 l'indice k (mediante lo *shift* a sinistra di una sola posizione) per ottenere l'Offset da sommare all'indirizzo Base dell'Array.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO Array testo[ ] di caratteri con codifica Unicode UTF-16, INDICE COSTANTE:

val = testo[5] ;

scegliendo i registri:

Base testo[ ]	val
$s0	$t1

traduzione in Assembly MIPS:

```
lhu  $t1,  10 ($s0)        #   val = testo[5]    Offset = 2 x 5 = 10
```

ESEMPIO Array testo[ ] di caratteri con codifica Unicode UTF-16, INDICE VARIABILE:

val = testo[k] ;

testo[k] = val ;

scegliendo i registri:

Base testo[ ]	val	k
$s0	$t1	$t3

la traduzione in Assembly MIPS è:

```
sll  $t5,  $t3,  1        #  $t5  contiene l'Offset = 2 x k
add  $t6,  $t5, $s0       # $t6 contiene l'indirizzo completo di testo[k]
lhu  $t1,  0 ($t6)        #   val = testo[k]
sh  $t1,  0 ($t6)         #   testo[k] = val
```

Il Linguaggio Java utilizza la codifica Unicode a 16 bit ed implementa il Tipo di dato *stringa* con la classe S*tring*, quindi le stringhe sono oggetti e gli operatori sono metodi della classe.

Il Linguaggio C++ supporta entrambe le implementazioni.

Nei Linguaggi C, C++ e Java una costante di Tipo *stringa* è rappresentata racchiudendola tra doppi apici:

```
char   testo[ ] =  "esempio"   ;
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.
- J. Lewis, W.Loftus, "Java software solutions", Addison Wesley.
- S.B. Lippman, J. Lajolie, C++ corso di programmazione, Addison Wesley.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).